

Alberta Buck – Ethereum Smart Contracts

Perry Kundert

March 9, 2026

1 Overview

Ethereum implementation of the Alberta Buck – citizen-issued, wealth-backed liquidity for Alberta. Three interacting smart contracts:

BuckCredit (ERC-721) An NFT representing an insurer’s offer of parametric insurance on a real-world asset. Supports deterministic depreciation (none, linear, declining balance) and piecemeal client activation.

Buck (ERC-20) A fungible token minted against aggregated BuckCredit values. Enforces per-account credit limits (credit value * BUCK_K) and collects quadratically-scaled default insurance premiums.

BuckKController An on-chain PID controller that computes a dynamic credit-limit multiplier from commodity-basket oracle prices vs. BUCK market price, maintaining purchasing-power parity.

Design document: The Alberta Buck – Ethereum Implementation

2 Getting Started

2.1 Prerequisites

- Nix (with flakes enabled) – provides Foundry, Solidity compiler, Python, Node.js
- An Ethereum RPC endpoint for forked-network testing (Alchemy free tier, Infura, or a local node)

2.2 Setup

```
git clone git@github.com:pjkundert/alberta-buck.git
cd alberta-buck
make nix-install          # Install Solidity dependencies (OpenZeppelin, Chainlink, Un
```

2.3 Build and Test

The `nix-%` Makefile target runs any target inside the Nix flake shell, providing Foundry (forge, anvil, cast), Solidity compiler, Python and Node.js – no manual `nix develop` required:

```
make nix-build            # Compile contracts
make nix-test            # All tests, verbose
make nix-snapshot        # Gas usage report
make nix-fmt             # Format Solidity source
```

Or enter the shell interactively for ad-hoc commands:

```
nix develop              # Enter dev shell
forge test --match-test test_activate  # Specific test
forge test --match-contract BuckCredit # All tests in a contract
```

2.4 Forked Network Testing

Copy `.env.example` to `.env` and set your RPC endpoint:

```
cp .env.example .env
# Edit .env with your Alchemy/Infura key (or http://localhost:8545 for a local node)
```

Then:

```
make nix-fork-sepolia    # Start Anvil forking Sepolia (runs in foreground)
make nix-test-fork-sepolia # Run tests against forked Sepolia
make nix-fork-mainnet    # Fork mainnet (for DeFi integration testing)
```

Pin a specific block for deterministic tests:

```
FORK_BLOCK=12345678 make nix-fork-sepolia
```

2.5 Python: Oracle History and Visualization

The `alberta_buck` Python module queries Chainlink price feed oracles on Ethereum mainnet to retrieve historical commodity prices. This validates the on-chain data sources used by the BuckKController PID loop and provides visualization for analysis.

2.5.1 How it works

The test connects to an Ethereum mainnet RPC endpoint and reads Chainlink's `AggregatorV3Interface` price feeds for Gold (XAU/USD) and Silver (XAG/USD) – the same oracle contracts used by the Solidity `BuckKControllerForkTest`:

Feed	Mainnet Address
XAU/USD	0x214eD9Da11D2fbe465a6fc601a91E62EbEc1a0D6
XAG/USD	0x379589227b15F1a12195D3f2d90bBc9F31f95235

For each of the past 365 days, the test estimates the Ethereum block number at that point in time (using the ~12.05s average post-merge block interval) and calls `latestRoundData()` with that block as the call context. This returns the most recent oracle price update as of that block – no archive node special features required beyond standard `eth_call` with a block parameter.

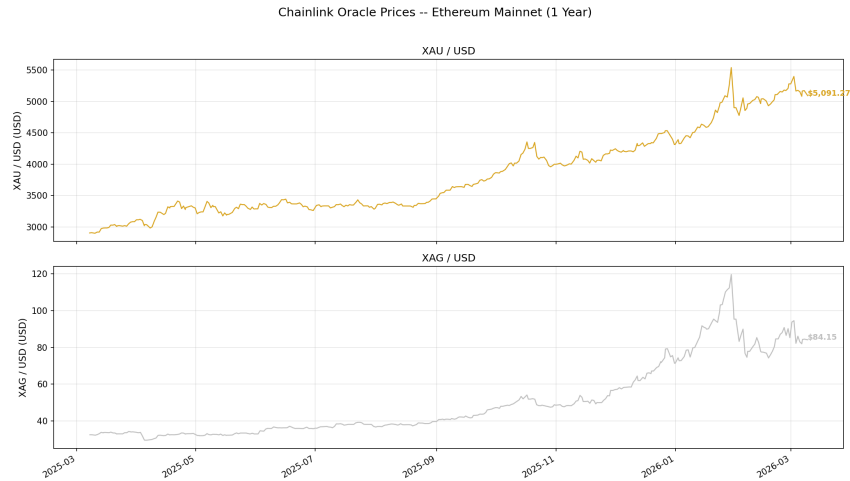
The result is 366 data points per feed (today plus 365 days back), which are plotted with matplotlib and saved as a PNG.

2.5.2 Running

Requires `MAINNET_RPC_URL` (or `ETH_RPC_URL`) pointing to an Ethereum mainnet RPC endpoint. Alchemy and Infura free tiers support the required historical `eth_call` queries.

```
export MAINNET_RPC_URL=https://eth-mainnet.g.alchemy.com/v2/YOUR_KEY
make nix-test-python
```

The test takes roughly 10-15 minutes (732 RPC calls: 366 days x 2 feeds). On completion it writes `alberta_buck/test/gold_silver_history.png`:



2.5.3 Future commodity oracles

The `alberta_buck.oracle` module is designed to support additional feeds. The BUCK commodity basket targets five components – gold, silver, oil, natural gas, and copper – with planned expansion to beef, lumber, electricity, and labour. For commodities without existing Chainlink mainnet feeds, custom oracle contracts will be deployed and populated with historical trend data sourced from public commodity price datasets.

2.6 Deploy (Local)

```
make nix-anvil &           # Start local Anvil node
make nix-deploy-local      # Deploy all three contracts
```

3 Project Structure

Path	Contents
<code>src/BuckCredit.sol</code>	ERC-721 insured asset NFT
<code>src/Buck.sol</code>	ERC-20 token with credit-limit minting
<code>src/BuckKController.sol</code>	PID value stabilization controller
<code>test/BuckCredit.t.sol</code>	Forge tests (depreciation, activation, updates)
<code>test/BuckKController.t.sol</code>	PID controller unit + mainnet fork tests
<code>script/Deploy.s.sol</code>	Deployment script
<code>alberta_buck/oracle.py</code>	Chainlink oracle historical price reader
<code>alberta_buck/test/</code>	Python tests (oracle history, visualization)
<code>pyproject.toml</code>	Python project configuration
<code>flake.nix</code>	Nix dev environment
<code>foundry.toml</code>	Forge configuration and remappings

4 License

GPL-3.0-or-later